INF211

LABORATORY LEAFLET

FOR STUDENTS

LABORATORY-6
Python Classes and Inheritance

Exercises	Explanations
1	Getter & Setter
2	Integer only Getter & Setter
3	Rectangle
4	Grades
Tasks	Explanations
1	Movie
2	Movie Catalog
3	Encoder

Exercise 1: In this function, write a class with the following properties and return the class. Your function should not take any parameters. Check the example in the provided submission python file.

Class properties:
- Class name should be ex1.
- Class should take one parameter (x), and should define and set an internal variable.

Class methods:
- get_value() that will return the internal variable value when called.
- set_value(x) that will update the internal variable with the given value.

Class constraints:
- inputs : $x \in \mathbb{Z}$

```
>>> A= exersice1()

>>> ainst = A(3)

>>> ainst.get_value()

3

>>> A = exersice1()

>>> ainst = A(1)

>>> ainst.get_value()

1

>>> A = exersice1()

>>> ainst = A(-4)

>>> ainst.get_value()

-4

>>> ainst.set_value(5)

>>> ainst.get_value()

5
```

Exercise 2: In this function, write a class with the following properties and return the class. Your function should not take any parameters.

Class properties:

● Class name should be ex2.

● Class should take one parameter (x), and should define and set an internal variable. Only integers should be allowed. If the passed parameter is any other type, it should set the internal variable to 0.

Class methods:

● get_value() that will return the internal variable value when called.

● set_value(x) that will update the internal variable with the given value. Only integers should be allowed. If the passed parameter is any other type, it should just ignore silently.

Class constraints:

● inputs : $x \in \mathbb{Z}$

Example Doctests:

```
>>> A = exercise2()

>>> ainst = A(3.5)

>>> ainst.get_value()

0

>>> A = exercise2()

>>> ainst = A([1])

>>> ainst.get_value()

0

>>> A = exercise2()

>>> ainst = A(None) # Creating an instance with None as parameter

>>> ainst.get_value()

0

>>> A = exercise2()

>>> ainst = A(-4)

>>> ainst.get_value()

-4

>>> ainst.set_value(7.6)
```

```
>>> ainst.get_value()

-4

>>> ainst.set_value(3)

>>> ainst.get_value()

3
```

Exercise 3: In this function, write a class with the following properties and return the class. Your function should not take any parameters.

Class properties:

● The purpose of the class is to define a rectangle with basic properties such as *area* and *perimeter*.

● Class name should be ex3.

● Class should take two parameters (x, y), and should define and set internal variables. These are the sides of the rectangle.

Class methods:

● get_area() that will return the *area* of the rectangle.

● get_perimeter() that will return the *perimeter* of the rectangle.

Class constraints:

● inputs : $\{x, y\} \in \mathbb{Z}$

```
Example Doctests:

>>> A = exercise3()

>>> ainst = A(3, 5)

>>> ainst.get_area()

15

>>> ainst.get_perimeter()

16
```

Exercise 4: In this function, write a class with the following properties and return the class. Your function should not take any parameters.

Class properties:

● The purpose of the class is to define a list of grades and calculate and hold the basic properties such as minimum, maximum, median and mean grades.

● Class name should be Grades.

● Class should not take any parameter, however should initialize internal values to 0.0.

Class methods:

● add_grade(x) that will add the given grade to the internal list.

● remove_grade(x) that will return the grade from the internal list if it exists, if it does not exist, it will just ignore it.

● get_min() return the minimum grade.

● get_max() return the maximum grade.

● get_mean() return the mean(average) grade.

● get_median() return the median grade. If the number of grades are even, returns the average of the middle two grades (assuming the grades are sorted).

Class constraints:

● inputs : None

```
Example Doctests:
>>> A = exercise4()
>>> ainst = A()
>>> ainst.get_min()
0.0
>>> ainst.get_max()
0.0
>>> ainst.get_mean()
0.0
>>> ainst.get_median()
0.0
>>> ainst.add_grade(3)
>>> ainst.add_grade(5)
>>> ainst.get_min()
3.0
>>> ainst.get_max()
5.0
>>> ainst.get_mean()
4.0
>>> ainst.get_median()
4.0
>>> ainst.add_grade(10)
```

```
>>> ainst.get_min()
3.0
>>> ainst.get_max()
10.0
>>> ainst.get_mean()
6.0
>>> ainst.get_median()
5.0
>>> ainst.remove_grade(15)
>>> ainst.remove_grade(10)
>>> ainst.get_min()
3.0
>>> ainst.get_max()
5.0
>>> ainst.get_mean()
4.0
>>> ainst.get_median()
4.0
```

Task 1a: In this function, write a class with the following properties and return the class. Your function should not take any parameters.

Class properties:

- The purpose of the class is to define a movie and get its properties.
- Class name should be Movie.
- Class should take mandatory and optional parameters. Mandatory ones are movie_name, director, and year, optional parameters are rating and length.
    - movie_name is a string
    - director is a string
    - year is a number between [1920 - 2021]
    - rating is a float. Should be between [0.0 - 10.0]. Other inputs should be ignored. Default is 0 mins. Default is 0.0
    - length is an integer in minutes. Should be between [0 - 500]. Other inputs should be ignored. Default is 0 mins.

Class methods:

- get_movie_name() that will return the movie name.
- get_director() that will return the director name.
- get_year() that will return the year that the movie is made.

● get_rating() return the movie rating.

● get_length() return the movie length.

● set_rating(x) set the movie rating. Should be a float and between [0.0 - 10.0]. Other inputs should be ignored.

● set_length(x) set the movie length. Should be an integer and between [0 - 500]. Other inputs should be ignored.

```
Example Doctests:
>>> Movie = task1a()
>>>film = Movie("Inception", "Christopher Nolan", 2010, 8.8, 148)
>>> film.get_movie_name()
Inception
>>> film.get_director()
Christopher Nolan
>>> film = Movie("The Shawshank Redemption", "Frank Darabont", 1994, 9.3, 142)
film.get_rating()
9.3
```

Task1b: In this task, you will practice inheritance by creating two derived classes that extend the Movie base class returned by task1a(). Your function must be named task1b, take no parameters, and return the classes.

Function requirements

- Function name: task1b
- The function must not take any parameters.
- Inside task1b, obtain the base class with: Movie = task1a()
- Do not modify the Movie class from Task 1a.
- Define and return two subclasses: Documentary(Movie) and ScienceFiction(Movie).

Class 1: Documentary(Movie)

Class properties

- Purpose: represent documentary movies as a specialization of Movie.
- Inherits all Movie properties: movie_name, director, year, rating, length.
- Adds one additional property: subject (string).

Constructor

- Mandatory parameters: movie_name, director, year, subject
- Optional parameters: rating = 0.0, length = 0
- subject must be a string; otherwise store it as an empty string "".

Class methods

- get_subject() - returns the documentary subject.
- get_info() - returns a single string containing all information (Movie fields + subject).
- matches_subject(keyword) - returns True if keyword (string) appears in subject (case-insensitive), otherwise False.

Class 2: ScienceFiction(Movie)

Class properties

- Purpose: represent science fiction movies as a specialization of Movie.
- Inherits all Movie properties: movie_name, director, year, rating, length.
- Adds additional properties:
  - subgenre (string, default: "General")
  - has_time_travel (boolean, default: False)

Constructor

- Mandatory parameters: movie_name, director, year
- Optional parameters: subgenre = "General", has_time_travel = False, rating = 0.0, length = 0
- subgenre must be a non-empty string; otherwise keep "General".
- has_time_travel must be boolean; otherwise keep False.

Class methods

- get_subgenre() - returns the subgenre.
- set_subgenre(subgenre) - updates subgenre if input is a non-empty string; otherwise ignore.
- set_time_travel(flag) - updates has_time_travel if input is boolean; otherwise ignore.
- is_mind_bending() - returns True if rating > 8.0, otherwise False.
- get_info() - returns a single string containing all information (Movie fields + sci-fi fields + a tag).
- recommended_audience() - returns: "Advanced" if mind-bending; "Intermediate" if not mind-bending but time travel is True; otherwise "Beginner".

```
Example Doctests:

>>> Movie, Documentary, ScienceFiction = task1b()
```

```
>>> d = Documentary("Free Solo", "Vasarhelyi", 2018,
"Climbing", 8.2, 100)
>>> d.get_subject()
Climbing
>>> d.matches_subject("climb")
True
>>> "Subject:" in d.get_info()
True

>>> sf = ScienceFiction("Interstellar", "Christopher Nolan",
2014, "Space Opera", True, 8.6, 169)
>>> sf.is_mind_bending()
True
>>> sf.recommended_audience()
Advanced
>>> "TimeTravel:" in sf.get_info()
True

>>> sf2 = ScienceFiction("Moon", "Duncan Jones", 2009,
"Psychological", False, 7.8, 97)
>>> sf2.is_mind_bending()
False
>>> sf2.recommended_audience()
Beginner
```

Task 2 : In this function, write a class with the following properties and return the class. Your function should not take any parameters.

Class properties:

● The purpose of the class is to define a movie catalog using the movie class that we defined in Task 1a.

○ You can either re-implement the Movie class here, or call task1a() within Task 2 to access that class.

● Class name should be MovieCatalog.

● Class should take one parameter called filename, which will point to a txt file with a list of movies. You should parse this file, and create a Movie class instance for each movie and add it to an internal list. (Thus **the list should hold a bunch of Movie class instances).**

Class methods:

● **add_movie(movie_name, director, year, rating=0.0, length=0)** will add the given movie to the list **if it is not already in the list**. (Do not forget to create a new Movie instance for this movie.)

● **remove_movie(movie_name)** that will remove the movie from the list if it exists. Ignore otherwise.

● **get_oldest()** that will return the **name** of the oldest movie in the list.

● **get_lowest_ranking()** that will return the **name** of the lowest ranking movie in the list.

● **get_highest_ranking()** that will return the **name** of the highest ranking movie in the list.

● **get_by_director(director)** that will return **a list of movies** that are made by the given director. If the director has no movies in the movie list, return an empty list.

```
Example Doctests:

>>> A = task2()

>>> ainst = A('movies.txt')

>>> ainst.add_movie('The Usual Suspects', 'Bryan Singer', 1995,
8.5, 106)

>>> ainst.get_oldest()

'The Godfather'

>>> ainst.get_highest_ranking()

'The Shawshank Redemption'

>>> ainst.get_lowest_ranking()

'The Usual Suspects'

>>> ainst.get_by_director('Nuri Bilge Ceylan')

[]

>>> ainst.get_by_director('Francis Ford Coppola')

['The Godfather', 'The Godfather: Part II']
```

Task 3: In this function, write a class with the following properties and return the class. Your function should not take any parameters.

Class properties:

● The purpose of the class is to apply given encodings on a string.

● Class name should be Encoder.

● Class should take one parameter (x) that will be used as the source string.

- Class should accept only alphanumeric characters. {a-z,A-Z,0-9}. Other characters should be ignored.

Class methods:

● **__str__** to print the stored string.

● **morse()** that will return the morse encoded version **as a list of strings**. Conversion table is given above. Encode long dashes with a dash character, and dots with a dot character.

● **binary()** that will return the 7-bit ascii binary encoded version **as a string**. [1]

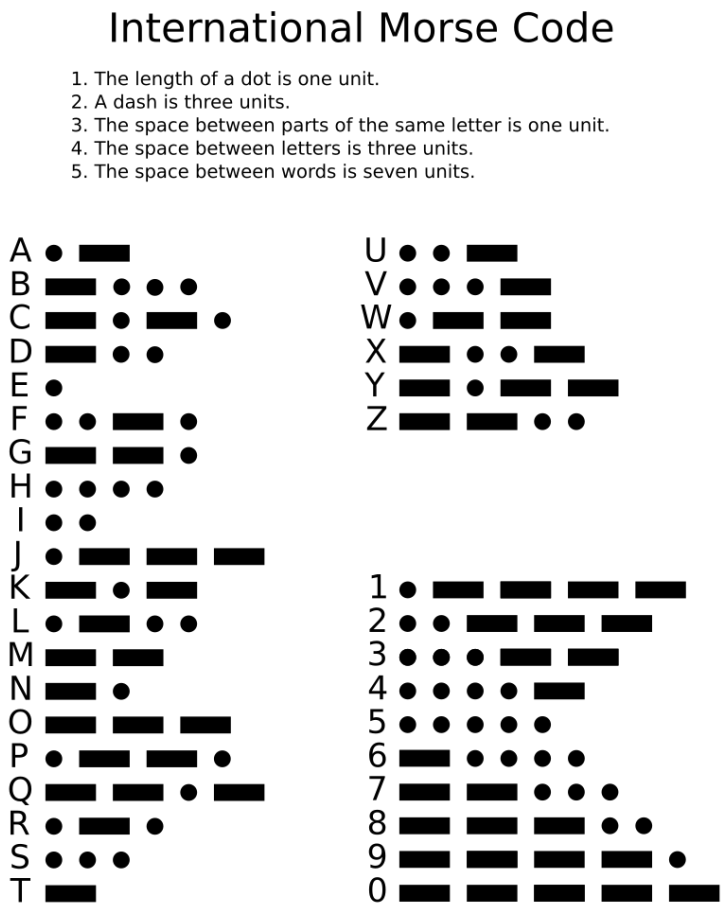
● **hex()** that will return the hex encoded version as a string. [1]

Class constraints:

● inputs : {x} ∈ english letters and numbers.

```
>>> A = task3()
>>> ainst = A("test")
>>> print(ainst)
test
>>> ainst.morse()
['-', '.', '...', '-']
>>> ainst.binary()
'1110100110010111100111110100'
>>> ainst.hex()
'74657374'
>>> ainst = A("çELık-!")
>>> print(ainst)
```

```
ELk

>>> ainst.morse()

['.', '.-..', '-.-']

>>> ainst.binary()

'10001011001100110101 1'

>>> ainst.hex()

'454c6b'
```



[1] https://www.asciitable.com